

Introducción al uso de registros nacionales de salud para investigación

Daniel R. Witte

Contents

Bienvenida	5
Datos clave	5
Público	5
Preparación	5
Re-utilización y licencia	5
Como se construyó este taller	5
.	5

I

Resumen

1	Temario	9
1.1	Objetivos de aprendizaje	9
2	¿Este taller es como para mi?	11
3	Horario	13
3.1	10 de septiembre 2026	13

II

Pre-taller

4	Tareas preparatorias antes del taller	17
5	Instalación de los programas	19
5.1	Windows	19
5.2	MacOS	19
5.3	Linux (Ubuntu)	19
6	Instalación de los paquetes R necesarios	21

III

Sesiones

7	Introducción al taller	25
7.1	Tema de esta presentación	25
7.1.1	Objetivos de aprendizaje	25
8	El potencial de los registros	27
8.1	Objetivos de aprendizaje	27

9	Partiendo de datos crudos	29
9.1	Objetivos de aprendizaje	29
9.2	Explorando el paquete OSDC	29
9.3	Registros y variables	30
9.4	Simulación de datos	30
9.5	Datos crudos	30
9.5.1	Preparación de los datos	30
9.5.2	Hacer correr la función de clasificación	31
9.5.3	Transferencia de los datos a formato R	32
9.5.4	Acerca de <code>stable_inclusion_date</code> y <code>raw_inclusion_date</code>	33
9.5.5	Borrar variables ya usadas	33
9.6		33
10	Análisis sencillo con datos de registro	35
10.1	Objetivos de aprendizaje	35
10.2	Datos analizables	35
10.2.1	Calculando la edad	35
10.2.2	Calculando la duración de diabetes	35
10.2.3	Calculando la edad al diagnóstico	35
10.2.4	Suprimiendo errores	36
10.2.5	Modificando algunas variables	36
10.3		36
10.4	Modelo mínimo de incidencia	36
11	Los datos de partida y la definición del seguimiento	37
12	Construir el objeto Lexis	39
12.1	Introducir el estado de diabetes con <code>cutLexis()</code>	39
12.2	El diagrama de estados	40
13	Diagramas de Lexis	41
14	Tasas de incidencia por sexo	43
14.1	Tabulación	43
14.2	Las estimaciones de tasa	43
14.3	El gráfico	43
14.4	La razón de tasas	44
15	Hacia dónde se generaliza esto	45
16	Errores frecuentes	47
17		49

Bienvenida



Bienvenidos al curso “Introducción al uso de registros nacionales de salud para investigación”. Este curso breve tiene como objetivo introducir los conceptos centrales del uso de registros de salud para investigación epidemiológica. El curso está enfocado en las enfermedades crónicas, especialmente en la diabetes pero los principios generales pueden aplicarse en otros contextos.

El curso se organiza por primera vez en 2026 en colaboración entre el [Grupo Crónicas](#), el [NCD Policy Lab](#) y [Steno Diabetes Center Aarhus](#).

Esta página web contiene todo el material del taller, incluyendo presentaciones y ejercicios. Está estructurado como un libro, con un “capítulo” para cada sesión. Haremos uso intensivo del internet durante el taller, donde las sesiones interactivas de programación en vivo siguen casi idénticamente los pasos en ésta página.

Datos clave

 10 de septiembre 2026

 9:00 - 12:00

 Universidad Peruana Cayetano Heredia, sede Miraflores, Lima, Perú

Público

El curso está diseñado como taller para investigadores jóvenes y estudiantes con conocimientos en análisis de datos interesados en conocer metodologías de análisis con registros nacionales de salud.

Preparación

Para agilizar las sesiones el día del taller, les pedimos que completen las [tareas preparatorias](#) antes de llegar a la clase.

Re-utilización y licencia

El taller se publica con una licencia [Licencia Internacional Pública de Atribución/Reconocimiento 4.0 de Creative Commons](#). Quiere decir que los materiales pueden ser utilizados, re-utilizados y adaptados, con tal que cualquier nueva versión o reproducción parcial o total contenga reconocimiento y atribución a éste material como fuente.

Como se construyó este taller

Los materiales de este taller fueron creados usando el formato [Quarto](#), [Git](#) y [GitHub](#) para la gestión del repositorio y Github Pages para crear y alojar la página web.

I

Resumen

1	Temario	9
1.1	Objetivos de aprendizaje	9
2	¿Este taller es como para mi?	11
3	Horario	13
3.1	10 de septiembre 2026	13

1. Temario

Este taller dura 3 horas y consiste de las siguientes sesiones:

Consulta también el [horario](#)

- [Introducción al taller](#)
- [Potencial de los registros](#)
- [Partiendo de los datos crudos](#)
- [Un análisis sencillo](#)
- [Cierre](#)

1.1 Objetivos de aprendizaje

Hemos diseñado el taller para enseñarte lo siguiente:

1. Primer objetivo

2. ¿Este taller es como para mí?

El curso está diseñado como taller para investigadores jóvenes y estudiantes con conocimientos en análisis de datos interesados en conocer metodologías de análisis con registros nacionales de salud.

Para ayudar a manejar las expectativas y para enfocar el desarrollo de los materiales, asumiremos cierta experiencia previa de los participantes de este taller. Esperamos que hayas estudiado, trabajado o tengas alguna experiencia con:

- Conceptos epidemiológicos básicos como prevalencia, determinantes, desenlaces, diseño de cohortes abiertas y cerradas, tiempo de seguimiento, censura, medidas de incidencia, medidas de riesgo, medidas de asociación, sesgos, confusión.
- Manejo de datos cuantitativos, conceptos básicos de la ciencia de datos como formatos *long* y *wide*, fusión de datos determinística y probabilística.
- Análisis de datos con programas estadísticos como R.

While we have these assumptions to help focus the content of the workshop, if you have an interest in the workshop but don't fit any of the assumptions, *you are still welcome to attend!* We welcome everyone until capacity has been reached.

In addition to the assumptions above, the workshop also has a fairly focused scope, which may also help you decide if this workshop is for you:

- TODO: List of concrete tasks we will do and not do.

3. Horario

Este taller contiene presentaciones, ejercicios participatorios con programación en vivo utilizando datos simulados, discusiones y trabajo en grupos.

3.1 10 de septiembre 2026

Horario	Módulo	Detalle	Expositor
09:00 - 09:10	Bienvenida	Presentación del docente y de las instituciones organizadoras.	
09:10 - 10:00	El potencial de los registros	Qué son los registros nacionales y poblacionales, y qué los hace tan valiosos. El modelo nórdico como ejemplo. Qué preguntas de investigación se vuelven posibles con estos datos que de otro modo serían muy difíciles. Fortalezas y limitaciones en lenguaje sencillo. Cierre del bloque contrastando con el contexto latinoamericano/peruano.	DW
10:00 - 10:15	Break		
10:15 - 11:00	De los datos crudos a una pregunta de investigación	Cómo se pasa de una base administrativa gigante a un conjunto de datos analizable. Ideas básicas: definir una población, elegir variables a partir de códigos, construir un seguimiento en el tiempo.	DW
11:00 - 11:50	Manos a la obra: un análisis sencillo	Un ejercicio guiado y simple con datos sintéticos o de ejemplo, donde todos siguen el mismo paso a paso: cargar los datos, mirar la estructura, y correr un análisis básico.	DW
11:50 - 12:00	Cierre. Ideas y discusión	Qué se necesitaría para hacer esto en nuestra	Todos

Horario	Módulo	Detalle	Expositor
		región/país y ronda de preguntas.	

II

Pre-taller

4	Tareas preparatorias antes del taller .	17
5	Instalación de los programas	19
5.1	Windows	19
5.2	MacOS	19
5.3	Linux (Ubuntu)	19
6	Instalación de los paquetes R necesarios	21

4. Tareas preparatorias antes del taller

Para agilizar el taller es importante que todos los participantes completen las siguientes tareas preparatorias:

1. Instala los programas R, RStudio, Quarto, y Git en sus versiones mas actuales de acuerdo con Chapter 5..
2. Instala los paquetes R necesarios.
3. Lee el temario (Chapter 1.).

Cada sección contiene detalles sobre cómo completar cada tarea.

 Tip

Puedes pasar a la siguiente sección haciendo click en la flechita en la parte inferior de cada página :

5. Instalación de los programas

Para poder participar en el taller, es importante que tengas los siguientes programas instalados en tu computadora. Es posible que ya hayas instalado algunos de ellos en el pasado. En ese caso por favor verifica que tengas instalada una versión mas reciente que la indicada en las instrucciones. Si no tendrás que actualizar las instalaciones.

5.1 Windows

1. **R**: Cualquier versión mas reciente que 4.6.0. Si has utilizado R anteriormente, puedes verificar tu versión actual dando éste comando en la consola: `R.version.string`.
2. **RStudio**: Cualquier versión mas reciente que 2026.08.2. Si has instalado RStudio anteriormente, puedes verificar la versión actual yendo al menu “Help -> About RStudio”.
3. **Git**: Escoge el enlace “Click here to download”. Usaremos Git durante algunas partes del taller. Al instalar Git, te pedirá seleccionar un “Text Editor” y aunque no usaremos esa funcion en el taller es necesario proporcionar el dato para la instalación de Git, así que lo mas fácil es escoger Notepad.

5.2 MacOS

1. **R**: Cualquier versión mas reciente que 4.6.0. Si has utilizado R anteriormente, puedes verificar tu versión actual dando éste comando en la consola: `R.version.string`. Si usas **Homebrew** (recomendado en general pero), la instalación de R is muy fácil. Solo hay que dar el siguiente comando en el Terminal de tu computadora:

```
brew install --cask r
```

2. **RStudio**: Cualquier versión mas reciente que 2026.08.2. Si has instalado RStudio anteriormente, puedes verificar la versión actual yendo al menu “Help -> About RStudio”. Con Homebrew:

```
brew install --cask rstudio
```

3. **Git**: Usaremos Git durante algunas partes del taller. MacOS viene con Git pre-instalado. Para instalarlo con Homebrew:

```
brew install git
```

5.3 Linux (Ubuntu)

1. **R**: Cualquier versión mas reciente que 4.6.0. Si has utilizado R anteriormente, puedes verificar tu versión actual dando éste comando en la consola: `R.version.string`.

```
sudo apt -y install r-base
```

2. **RStudio**: Cualquier versión mas reciente que 2026.08.2. Si has instalado RStudio anteriormente, puedes verificar la versión actual yendo al menu “Help -> About RStudio”.
3. **Git**: Usaremos Git durante algunas partes del taller.

```
sudo apt install git
```

Todos estos programas son necesarios para el taller, Git incluido. Git es usado para la gestión formal de versiones de archivos. Lo usamos porque tiene mucha documentación muy buena. Puedes aprender mas en éste libro en línea [Happy Git with R](#). Especialmente la sección “Why Git” explica bien por qué lo usamos. Usuarios de Windows a veces encuentran dificultades durante la instalación de Git. La siguiente guía ofrece pautas y ayuda [Installing Git for Windows](#).

6. Instalación de los paquetes R necesarios

Antes de iniciar la sesión debemos instalar los paquetes R necesarios para este taller:

```
install.packages("tidyverse") install.packages("osdc") install.packages("purrr") install.packages("Epi")  
install.packages("DBI") install.packages("duckdb") install.packages("knitr") install.packages("rmark-  
down") install.packages("ggplot2")
```


III

Sesiones

7	Introducción al taller	25
7.1	Tema de esta presentación	25
8	El potencial de los registros	27
8.1	Objetivos de aprendizaje	27
9	Partiendo de datos crudos	29
9.1	Objetivos de aprendizaje	29
9.2	Explorando el paquete OSDC	29
9.3	Registros y variables	30
9.4	Simulación de datos	30
9.5	Datos crudos	30
9.6		33
10	Análisis sencillo con datos de registro	35
10.1	Objetivos de aprendizaje	35
10.2	Datos analizables	35
10.3		36
10.4	Modelo mínimo de incidencia	36
11	Los datos de partida y la definición del seguimiento	37
12	Construir el objeto Lexis	39
12.1	Introducir el estado de diabetes con cutLexis()	39
12.2	El diagrama de estados	40
13	Diagramas de Lexis	41
14	Tasas de incidencia por sexo	43
14.1	Tabulación	43
14.2	Las estimaciones de tasa	43
14.3	El gráfico	43
14.4	La razón de tasas	44
15	Hacia dónde se generaliza esto	45
16	Errores frecuentes	47
17		49

7. Introducción al taller

7.1 Tema de esta presentación

Esta presentación da una introducción general a los conceptos centrales del trabajo con registros sanitarios. Se dicta como primera sesión o como presentación pre-curso (en línea).

7.1.1 Objetivos de aprendizaje

Después de esta presentación los estudiantes tendrán los siguientes conocimientos:

- Qué es un registro sanitario; con cuales componentes debe contar
- A qué tipo de preguntas puede responder un registro
- Cuales son los tipos de usuarios de los datos de un registro sanitario
- A qué niveles existen datos / registros sanitarios y cuales son las necesidades de información a cada nivel

8. El potencial de los registros

En esta sesión repasaremos rápidamente algunas de las definiciones

8.1 Objetivos de aprendizaje

El objetivo de esta sesión es que el estudiante conozca y pueda explicar:

- Qué son los registros sanitarios poblacionales, con enfoque en enfermedades crónicas
- Cómo se construyen y mantienen
- Qué los hace tan valiosos
- Cual es la experiencia de los países nórdicos y de Brasil

Tomando el modelo nórdico como ejemplo discutiremos qué tipo de preguntas de investigación se vuelven posibles con estos datos que de otro modo serían muy difíciles. Discutiremos fortalezas, limitaciones y retos. Cerraremos el bloque contrastando con el contexto latinoamericano/peruano.

9. Partiendo de datos crudos

9.1 Objetivos de aprendizaje

En esta sesión vamos a investigar la estructura de las bases de datos del registro de diabetes de Dinamarca, basándonos en datos simulados. Esto quiere decir que todos los datos que veremos son generados aleatoriamente y no pertenecen a ninguna persona real.

9.2 Explorando el paquete OSDC

Para éste taller usaremos el paquete “osdc” (Open Source Diabetes Classifier). Antes de iniciar esta sesión debes haber completado las tareas pre-taller, incluyendo la instalación de paquetes.

Podemos consultar la [documentación del paquete osdc](#) (en Inglés).

Para usar paquetes en una sesión debemos cargarlos:

```
library(osdc)
library(tidyverse)
```

```
— Attaching core tidyverse packages — tidyverse 2.0.0 —
✓ dplyr      1.2.1      ✓ readr      2.2.0
✓ forcats    1.0.1      ✓ stringr    1.6.0
✓ ggplot2     4.0.3      ✓ tibble     3.3.1
✓ lubridate  1.9.5      ✓ tidyr      1.3.2
✓ purrr       1.2.2
— Conflicts — tidyverse_conflicts() —
* dplyr::filter() masks stats::filter()
* dplyr::lag()     masks stats::lag()
! Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(purrr)
library(Epi)
library(DBI)
library(duckdb)
library(knitr)
library(rmarkdown)
library(ggplot2)
```

El paquete ‘osdc’ contiene varias funciones, pero también documentación en forma de guías o viñetas. Podemos revisarlas dentro del ambiente R.

La primera nos da la documentación del algoritmo

```
vignette("algorithm")
```

```
starting httpd help server ... done
```

El siguiente paso es revisar todas las fuentes de datos que son necesarias para crear el registro.

```
vignette("data-sources")
```

9.3 Registros y variables

Como se puede ver, hay 5 registros, pero en algunos casos un solo registro tiene varias tablas temáticas que se acceden como bases separadas (Landspatientregisterets) o separadas en el tiempo (Sygesikringsregisteret).

La segunda tabla nos muestra todas las variables de cada registro que son necesarias para construir el registro de diabetes.

Podemos consultar estos datos al detalle generando una variable que contiene la lista de registros.

```
reg_list <- names(registers())
print(reg_list)
```

```
[1] "bef"          "lmdb"          "lpr_adm"       "lpr_diag"
[5] "lpr3a_kontakt" "lpr3a_diagnose" "lpr3f_kontakter" "lpr3f_diagnoser"
[9] "sysi"         "sssy"         "lab_forsker"
```

9.4 Simulación de datos

Como vamos a trabajar con datos simulados, el siguiente paso es simular el registro de la población total.

```
sim_bef <- simulate_registers("bef", n = 10000)
sim_bef <- sim_bef$bef
head(sim_bef)
```

```
# A tibble: 6 × 3
  koen pnr      foed_dato
<int> <chr>   <date>
1     1 164409653234 1923-03-01
2     2 952443913885 1933-03-21
3     1 679714832266 1959-10-20
4     1 447393958677 1939-10-13
5     1 447559464315 2022-01-10
6     1 120172052967 2016-12-07
```

Después simulamos el registro de laboratorios.

```
sim_lab_forsker <- simulate_registers("lab_forsker", n = 10000)
sim_lab_forsker <- sim_lab_forsker$lab_forsker
head(sim_lab_forsker)
```

```
# A tibble: 6 × 4
  pnr      samplingdate analysiscode value
<chr>   <date>             <chr>      <dbl>
1 164409653234 2017-11-05 NPU15168    86.4
2 952443913885 2011-09-25 NPU86660    1.43
3 679714832266 2021-10-15 NPU27300    74.9
4 447393958677 2022-12-06 NPU27300   111.
5 447559464315 2025-12-10 NPU44101    70.1
6 120172052967 2011-12-21 NPU60312    43.9
```

9.5 Datos crudos

9.5.1 Preparación de los datos

El paquete requiere que los datos estén en formato [DuckDB](#). Esta es una base de datos de alta performance que puede manejar volúmenes grandes de datos sin que sea necesario cargarlos todos a memoria en una sola vez.

Para este ejemplo generaremos datos simulados del formato de registro con el comando `simulate_registers()` y luego los convertiremos al formato DuckDB:

```
register_data <- registers() |>
  names() |>
  simulate_registers(n=10000) |>
  purrr::map(duckplyr::as_duckdb_tibble) |>
  # Convert to a DuckDB connection, as duckplyr is still
  # in early development, while the DBI-DuckDB connection
  # is more stable.
  purrr::map(duckplyr::as_tbl)
```

```
duckdb keeps downloaded extensions and secrets in a temporary directory:
i /tmp/RtmpkxWAhS/duckdb
This is removed when the R session ends.
• Extensions are re-downloaded each session.
• Secrets are lost.
i Run duckdb(shared_home = TRUE) (or create ~/.duckdb) to keep them (suitable for
most users).
i Run duckdb(shared_home = FALSE) to accept the temporary directory (and silence
this message).
i See ?duckdb_storage for details and alternatives.
```

El resultado es una lista con nombres, donde cada elemento corresponde a un registro en forma de una tabla de DuckDB. Los datos del LPR abarcan múltiples versiones (LPR2 y LPR3), que deben prepararse y unirse en una única tabla antes de utilizarse como entrada para `classify_diabetes()`. Lo mismo ocurre con los registros de los servicios de salud (SSSY y SYSI). **osdc** proporciona funciones auxiliares para realizar esta tarea.

```
lpr <- list(
  prepare_lpr2(register_data$lpr_adm, register_data$lpr_diag),
  prepare_lpr3f(register_data$lpr3f_kontakter, register_data$lpr3f_diagnoser),
  prepare_lpr3a(register_data$lpr3a_kontakt, register_data$lpr3a_diagnose)
) |>
  join_registers()

hsr <- list(register_data$sssy, register_data$sysi) |> join_registers()
```

The remaining registers are extracted from the `register_data` list, so they are ready to be passed as arguments to `classify_diabetes()`:

Los registros restantes se extraen de la lista `register_data`, y así quedan listos para pasarlos como argumentos a `classify_diabetes()`.

```
bef <- register_data$bef
lmdb <- register_data$lmdb
lab_forsker <- register_data$lab_forsker
```

9.5.2 Hacer correr la función de clasificación

Ahora estamos listos para correr el algoritmo de clasificación. Pasamos todos los registros a `classify_diabetes()`:

```
classified_diabetes <- classify_diabetes(
  lpr = lpr,
  hsr = hsr,
  lab_forsker = lab_forsker,
  bef = bef,
  lmdb = lmdb
)

classified_diabetes
```

```
# A query: ?? x 5
# Database: DuckDB 1.5.5 [unknown@Linux 6.17.0-1022-azure:R 4.6.1//tmp/RtmpkxWAhS/
duckplyr/duckplyr21c5794d1883.duckdb]
  pnr          stable_inclusion_date raw_inclusion_date has_t1d has_t2d
  <chr>         <date>              <date>          <lgl>   <lgl>
1 000968900938 2017-02-13            2017-02-13      FALSE   TRUE
2 002118745152 2011-10-30            2011-10-30      FALSE   TRUE
3 020930073909 2007-11-26            2007-11-26      FALSE   TRUE
4 023901517086 2025-03-24            2025-03-24      FALSE   TRUE
5 034983397970 2022-10-09            2022-10-09      FALSE   TRUE
6 036188275183 2024-07-25            2024-07-25      FALSE   TRUE
7 079844973894 2011-01-20            2011-01-20      FALSE   TRUE
8 089028788111 2024-11-08            2024-11-08      FALSE   TRUE
9 134624431203 2025-01-17            2025-01-17      FALSE   TRUE
10 175783911906 2025-11-19            2025-11-19      FALSE   TRUE
# i more rows
```

Como se puede ver, este proceso devuelve una tabla de DuckDB con los individuos clasificados como personas con diabetes tipo 1 (DM1) o diabetes tipo 2 (DM2), junto con la fecha de la clasificación. Cada una de las columnas de la salida se explica en la Section 9.5.3.1 más adelante.

9.5.3 Transferencia de los datos a formato R

Por ahora los datos están guardados en formato DuckDB, lo que quiere decir que no han sido cargados directamente a la memoria de R. Para transferir los datos a R usamos `dplyr::collect()`:

```
classified_diabetes <- classified_diabetes |>
  dplyr::collect()

classified_diabetes
```

```
# A tibble: 954 x 5
  pnr          stable_inclusion_date raw_inclusion_date has_t1d has_t2d
  <chr>         <date>              <date>          <lgl>   <lgl>
1 000968900938 2017-02-13            2017-02-13      FALSE   TRUE
2 002118745152 2011-10-30            2011-10-30      FALSE   TRUE
3 020930073909 2007-11-26            2007-11-26      FALSE   TRUE
4 023901517086 2025-03-24            2025-03-24      FALSE   TRUE
5 034983397970 2022-10-09            2022-10-09      FALSE   TRUE
6 036188275183 2024-07-25            2024-07-25      FALSE   TRUE
7 079844973894 2011-01-20            2011-01-20      FALSE   TRUE
8 089028788111 2024-11-08            2024-11-08      FALSE   TRUE
9 134624431203 2025-01-17            2025-01-17      FALSE   TRUE
10 175783911906 2025-11-19            2025-11-19      FALSE   TRUE
# i 944 more rows
```

Ahora vemos que en los datos simulados 954 individuos han sido clasificados con un diagnóstico de diabetes.

9.5.3.1 Cómo entender los datos

El resultado es una tabla con una fila por individuo clasificado y cinco columnas:

Columna	Descripción
<code>pnr</code>	El número identificador individual pseudonimizado.
<code>stable_inclusion_date</code>	La fecha del segundo evento de inclusión, si cae después de <code>stable_inclusion_start_date</code> (valor predeterminado: 1998). <code>NA</code> para fechas anteriores.

Columna	Descripción
<code>raw_inclusion_date</code>	La fecha del segundo evento de inclusión sin imponer <code>NA</code> para fechas anteriores a <code>stable_inclusion_start_date</code> .
<code>has_t1d</code>	<code>TRUE</code> si el individuo se clasifica como caso de diabetes tipo 1, <code>FALSE</code> en otros casos.
<code>has_t2d</code>	<code>TRUE</code> si el individuo se clasifica como caso de diabetes tipo 2, <code>FALSE</code> en otros casos.

i Note

9.5.4 Acerca de `stable_inclusion_date` y `raw_inclusion_date`

La `raw_inclusion_date` corresponde simplemente a la fecha del segundo evento que cumple los criterios de clasificación. Sin embargo, para los eventos anteriores a 1998, los datos de los registros pueden no tener una cobertura suficiente para distinguir de forma fiable entre casos nuevos (incidentes) y casos ya existentes (prevalentes). Por ello, la columna `stable_inclusion_date` se establece como `NA` para esas fechas más tempranas, con el fin de señalar esta incertidumbre.

La función `classify_diabetes()` incluye el parámetro `stable_inclusion_start_date`, cuyo valor predeterminado es `01-01-1998`. Esto significa que es posible modificar la fecha a partir de la cual la clasificación se considera estable.

Véase la sección ‘Interface’ bajo `vignette("design")` para mayor información sobre los datos resultantes.

9.5.5 Borrar variables ya usadas

Antes de continuar vamos a hacer algo de limpieza, eliminando las variables que ya no vamos a necesitar.

```
rm(bef, hsr, lab_forsker, lmdb, lpr, register_data, reg_list)
```

9.6

10. Análisis sencillo con datos de registro

10.1 Objetivos de aprendizaje

En esta sesión vamos a continuar explorando la base de datos que hemos generado simulando el registro de diabetes de Dinamarca

10.2 Datos analizables

Ahora que tenemos las fechas de incidencia de diabetes en 'classified_diabetes' y la población total en 'sim_bef' (sexo y fecha de nacimiento) podemos generar una base que contenga toda la información ligando estas 2 bases. Lo hacemos de forma determinística, con la clave del PNR.

```
sim_diab <- right_join(classified_diabetes, sim_bef, by = "pnr")
```

10.2.1 Calculando la edad

El primer paso sigue siendo preparatorio; vamos a calcular la edad.

```
sim_diab$age <- as.numeric(today() - sim_diab$foed_dato)
head(sim_diab$age)
```

Nos damos cuenta de que la edad está expresada en días, así que tenemos que transformarla en años

```
sim_diab$age <- sim_diab$age / 365.25
head(sim_diab$age)
hist(sim_diab$age)
```

Podemos ver claramente que estos datos son simulados porque observamos algunas edades extremas y además vemos que la distribución es casi uniforme mientras que sería mucho más probable ver una distribución que aproxime la pirámide poblacional de Dinamarca.

10.2.2 Calculando la duración de diabetes

Del mismo modo vamos a calcular la duración de la diabetes para casos con el diagnóstico.

```
sim_diab$duration <- as.numeric(
  (today() - sim_diab$raw_inclusion_date) / 365.25
)
head(sim_diab$duration)
hist(sim_diab$duration)
```

10.2.3 Calculando la edad al diagnóstico

```
sim_diab$age_dm <- as.numeric(
  sim_diab$stable_inclusion_date - sim_diab$foed_dato
) /
  365.25
head(sim_diab$age_dm)
hist(sim_diab$age_dm)
```

10.2.4 Suprimiendo errores

Nos damos cuenta que hay valores negativos en la edad al diagnóstico a consecuencia del uso de datos simulados. Para poder progresar en el análisis vamos a suprimir los casos con edades al diagnóstico negativas. En casos reales también vale la pena verificar esto ya que puede haber errores administrativos o de la base de datos.

```
sim_diab <- sim_diab[-which(sim_diab$age_dm <= 0), ]
```

10.2.5 Modificando algunas variables

Ya que el modelo simulado da solo casos de diabetes tipo 2, vamos a suprimir la variable 'has_t1d'. Además vamos a suprimir la variable 'raw_inclusion_date' para el ejemplo de este taller.

```
sim_diab <- sim_diab |> select(!c(has_t1d, raw_inclusion_date))
```

Observamos que por el modo en que se creo la base actual (sim_diab) juntando dos bases anteriores, la variable 'has_t2d' actualmente tiene valores faltantes (NA) para los que no desarrollan diabetes, mientras que debería ser FALSE. Vamos a hacer el cambio:

```
sim_diab$has_t2d[is.na(sim_diab$has_t2d)] <- FALSE
```

Finalmente la variable a la variable sexo le falta codificación. Vamos a añadir una etiqueta:

```
sim_diab$sex <- factor(
  sim_diab$koen,
  levels = 1:2,
  labels = c("Male", "Female")
)
```

10.3

10.4 Modelo mínimo de incidencia

Vamos a continuar con un análisis de incidencia mínimo, con una sola transición. En este modelo todas las personas empiezan la vida en el estado **Healthy** y lo único que puede ocurrir es el paso al estado **T2D**. En los datos simulados no hay defunciones, de modo que no hace falta un planteamiento de riesgos competitivos. Aun así construiremos el análisis como un objeto *Lexis*, porque es la estructura de base del paquete *Epi* y va a servir para comprender el modelo de transición.

Los tres pasos son:

1. Reestructurar los registros individuales en un objeto *Lexis*, introducir el estado de diabetes y dividir el seguimiento en bandas de edad.
2. Dibujar diagramas de *Lexis* para ver cómo es realmente el seguimiento.
3. Tabular y graficar las tasas de incidencia específicas por edad en hombres y mujeres.

11. Los datos de partida y la definición del seguimiento

Partimos del data frame `sim_diab`, con una fila por persona:

Variable	Tipo	Significado
<code>pnr</code>	entero	identificador de la persona
<code>stable_inclusion_date</code>	Date	fecha del diagnóstico de T2D
<code>has_t2d</code>	lógico	¿recibió la persona un diagnóstico de T2D?
<code>koen</code>	factor	sexo, ya etiquetado
<code>foed_dato</code>	Date	fecha de nacimiento
<code>age</code>	numérico	edad actual — edad al final de la observación, en años
<code>duration</code>	numérico	duración de la diabetes en años

```
str(sim_diab)
```

La decisión de diseño de este tutorial es que **el seguimiento empieza en el nacimiento**. Cada persona entra en la cohorte a la edad 0, sana, y se la observa hasta su edad actual. Quien tiene `has_t2d == TRUE` pasa por un diagnóstico en algún punto intermedio.

En la escala de edad eso se ve así:

edad 0 ————— Healthy ————— edad al dx — T2D — edad actual
(en riesgo) (fuera de riesgo)

El tiempo transcurrido después del diagnóstico **no** está en riesgo de diabetes incidente, así que hay que excluirlo del denominador. Conseguir eso de forma automática es una de las razones principales para usar `Lexis` en lugar de tabular a mano.

⚠ Sobre el realismo del diseño

Seguir datos de registro desde el nacimiento suele ser imposible: las personas entran en observación cuando arranca el registro o cuando inmigran, y fingir lo contrario genera tiempo-persona inmortal. Aquí los datos están simulados con cobertura completa de toda la vida, así que no hay problema. En una cohorte real basada en el CPR se entraría en `pmax(foed_dato, inicio del registro, fecha de inmigración)`, lo que supone un `pmax()` adicional y no cambia nada de lo que sigue.

12. Construir el objeto Lexis

Un objeto `Lexis` es un data frame de *intervalos de seguimiento* en una o varias **escalas de tiempo**, más unas columnas de control:

- `lex.id` — identificador de la persona
- `lex.dur` — longitud del intervalo, en las unidades de las escalas de tiempo (aquí años)
- `lex.Cst` — estado al **inicio** del intervalo (*Current state*)
- `lex.Xst` — estado al **final** del intervalo (*eXit state*)

Todas las escalas de tiempo avanzan al mismo ritmo, de modo que se indica dónde *entra* cada persona en todas las escalas y dónde *sale* en una sola. `Epi` deduce el resto.

Usaremos dos escalas, siguiendo la convención habitual de `Epi`:

- `A` — edad de entrada en el seguimiento; todas las personas entran en 0 en este ejemplo
- `P` — tiempo calendario (*period*) en años decimales; `cal.yr()` convierte un `Date` a esa escala, que es lo que hace conmensurables el calendario y la edad

Como todo el mundo entra a la edad 0, la escala natural para especificar la salida es la edad: la salida ocurre a la edad actual de la persona, es decir, en `age`.

! Por qué las escalas se llaman `A` y `P` y no `age` y `per`

Cada escala de tiempo se convierte en una columna del objeto `Lexis` con ese mismo nombre. Si se llama `age` a la escala mientras el data frame ya tiene una columna `age`, `Lexis()` se detiene con:

```
Error in Lexis(...) : Cannot merge data with duplicate names:age
```

Hay dos salidas: renombrar la escala, o renombrar la columna de datos. Renombrar la escala es menos invasivo y coincide con la notación `A / P` que usan la documentación y las viñetas de `Epi`. La columna `age` sigue estando disponible en el objeto resultante con su significado original —edad al final de la observación—, mientras que la columna `A` guarda la edad al inicio de cada intervalo.

```
Lx <- Lexis(  
  entry = list(A = 0, P = cal.yr(foed_dato)),  
  exit = list(A = age),  
  entry.status = factor("Healthy", levels = c("Healthy", "T2D")),  
  exit.status = factor("Healthy", levels = c("Healthy", "T2D")),  
  id = pnr,  
  data = sim_diab  
)  
  
summary(Lx)  
head(Lx)
```

En este punto *nadie* tiene diabetes: cada intervalo es una vida completa desde 0 hasta la edad actual, con `lex.Cst == lex.Xst == "Healthy"`. `lex.dur` es igual a `age`, así que `sum(Lx$lex.dur)` son los años-persona totales de la cohorte.

12.1 Introducir el estado de diabetes con `cutLexis()`

`cutLexis()` corta el seguimiento de cada persona en un punto determinado y le cambia el estado a partir de ahí. Es la forma idiomática de añadir un estado intermedio a un objeto `Lexis` ya construido.

```
Dx <- cutLexis(  
  Lx,
```

```

cut = Lx$age_dm, # edad al diagnóstico; NA = sin corte
timescale = "A", # el punto de corte se da en la escala de edad
new.state = "T2D",
new.scale = "tfd", # opcional: añade la escala "tiempo desde el dx"
precursor.states = "Healthy" # estados que el nuevo estado sobrescribe
)

```

Tres detalles que conviene entender:

- `cut` se toma **en el orden de filas del objeto `Lexis`**, no en el de `sim_diab`. Hay que escribir siempre `Lx$age_dm` y no `sim_diab$age_dm`: si `Lexis()` descartó alguna fila, los dos vectores dejan de estar alineados y los cortes caen sobre las personas equivocadas.
- Un `NA` en `cut` significa “no cortar a esta persona”. Los no casos conservan un único intervalo.
- `new.scale = "tfd"` añade una escala de tiempo que vale `NA` antes del diagnóstico y cuenta desde 0 a partir de él. No hace falta para la incidencia, pero no cuesta nada y es la escala sobre la que se modelarían los desenlaces posteriores al diagnóstico.

```

summary(Dx)

table(table(Dx$lex.id)) # los casos tienen dos filas; los no casos, una
tapply(Dx$lex.dur, Dx$lex.Cst, sum) # años-persona repartidos entre los dos estados

```

La tabla de transiciones ya muestra `Healthy -> T2D`. El corte conserva los años-persona: parte las vidas, no las acorta, así que `sum(Dx$lex.dur)` sigue siendo igual que antes; lo que cambia es que ahora una parte pertenece al estado `T2D`.

12.2 El diagrama de estados

`boxes()` dibuja la estructura de estados con los años-persona, el número de transiciones y las tasas estimadas encima. Para un modelo de dos estados resulta casi trivial, pero es un buen reflejo: enseña qué modelo cree `Epi` que se ha especificado.

```

boxes(Dx, boxpos = TRUE, show.BE = TRUE, scale.R = 1000, hmult = 1.2)

```

`scale.R = 1000` pone la tasa de la flecha en casos por 1000 años-persona; `show.BE = TRUE` añade cuántas personas empiezan y terminan en cada estado.

13. Diagramas de Lexis

Un diagrama de Lexis representa el seguimiento como segmentos en el plano (tiempo calendario, edad). Como todo el mundo entra a la edad 0, cada persona es una única línea a 45° que arranca del eje horizontal en su año de nacimiento. Con una cohorte de tamaño registro esto se convierte en una mancha sólida, así que primero conviene muestrear un número manejable de personas.

```
ids <- sample(unique(Dx$lex.id), 60)
Dsub <- subset(Dx, lex.id %in% ids)
```

Los colores se derivan de los niveles de `koen`, de manera que el código funciona sea cual sea el idioma de las etiquetas ya presentes en los datos:

```
cols <- setNames(c("#1B6CA8", "#C1462F"), levels(sim_diab$koen))
cols
```

`plot.Lexis()` recibe las escalas de tiempo como primer argumento. Se pueden dar por posición (1:2 significa “primera escala en el eje x, segunda en el eje y”), pero como aquí queremos la orientación convencional —calendario en horizontal, edad en vertical— es más claro nombrarlas.

```
par(mar = c(4, 4, 1, 1), las = 1)

plot(
  Dsub,
  c("P", "A"),
  lwd = 1.5,
  col = c("Healthy" = "grey60", "T2D" = "#C1462F")[Dsub$lex.Cst],
  grid = TRUE,
  xlab = "Calendar year",
  ylab = "Age (years)"
)

# Punto final de cada intervalo: una cruz donde hubo diagnóstico
points(
  Dsub,
  c("P", "A"),
  pch = c("Healthy" = NA, "T2D" = 3)[Dsub$lex.Xst],
  col = "#C1462F",
  cex = 0.9,
  lwd = 2
)

legend(
  "topleft",
  bty = "n",
  legend = c("Seguimiento sano", "Tras el diagnóstico", "Diagnóstico de T2D"),
  col = c("grey60", "#C1462F", "#C1462F"),
  lty = c(1, 1, NA),
  lwd = 2,
  pch = c(NA, NA, 3)
)
```

Cada línea de vida es gris hasta el diagnóstico y roja después. Solo los tramos grises entran en el denominador de la incidencia.

Una segunda vista, coloreada por sexo en lugar de por estado:

```
par(mar = c(4, 4, 1, 1), las = 1)

plot(
  Dsub,
  c("P", "A"),
  lwd = 1.5,
  grid = TRUE,
  col = cols[Dsub$koen],
  xlab = "Calendar year",
  ylab = "Age (years)"
)

legend("topleft", bty = "n", legend = names(cols), col = cols, lwd = 2)
```

También existe `Lexis.diagram()` si se quiere una rejilla de Lexis vacía y formal para anotar a mano, pero para dibujar seguimiento real lo que se usa es `plot.Lexis()`.

14. Tasas de incidencia por sexo

Aquí vamos a calcular una sola tasa por sexo: los eventos y los años-persona en riesgo se suman sobre todo el rango de edad conservado y se dividen. Es la tasa cruda de la cohorte.

14.1 Tabulación

`stat.table()` es la herramienta de `Epi` para esto. `ratio(D, Y, 1000)` calcula la tasa por 1000 años-persona dentro de cada celda. Ahora el único índice es el sexo.

```
stat.table(
  index = list(Sex = koen),
  contents = list(
    "Person-years" = sum(lex.dur),
    "Cases" = sum(event),
    "Rate per 1000" = ratio(event, lex.dur, 1000)
  ),
  margins = TRUE,
  data = Rx
)
```

La columna de márgenes da la tasa global de la cohorte, que es la que se obtendría ignorando también el sexo.

14.2 Las estimaciones de tasa

La tasa empírica es D / Y , con un intervalo de confianza al 95 % aproximado $\text{rate} * \exp(\pm 1.96 / \sqrt{D})$. El error estándar de una tasa en escala logarítmica depende únicamente del número de eventos, así que con miles de casos el intervalo es estrecho.

```
rates <- aggregate(cbind(event, lex.dur) ~ koen, data = Rx, FUN = sum)

rates$rate <- rates$event / rates$lex.dur * 1000
rates$lo <- rates$rate * exp(-1.96 / sqrt(rates$event))
rates$hi <- rates$rate * exp(1.96 / sqrt(rates$event))

rates
```

14.3 El gráfico

Con una sola estimación por sexo no hay ninguna curva que dibujar, así que lo apropiado es un gráfico de puntos con sus intervalos. El eje logarítmico se mantiene: las tasas de incidencia se comparan de forma multiplicativa, y en escala logarítmica la distancia vertical entre los dos puntos es el logaritmo de la razón de tasas.

```
par(mar = c(4, 4.5, 1, 1), las = 1, bty = "n")

xpos <- seq_len(nrow(rates))

plot(
  NA,
  xlim = c(0.5, nrow(rates) + 0.5),
  ylim = range(rates$lo, rates$hi),
  log = "y",
```

```

xaxt = "n",
xlab = "",
ylab = "T2D incidence rate per 1000 PY"
)

segments(xpos, rates$lo, xpos, rates$hi, col = cols[rates$koen], lwd = 2)
points(xpos, rates$rate, pch = 16, cex = 1.8, col = cols[rates$koen])

axis(1, at = xpos, labels = as.character(rates$koen))

```

14.4 La razón de tasas

Leer la diferencia entre los dos puntos a ojo funciona, pero el modelo de Poisson la cuantifica con su intervalo de confianza en una línea. La familia `poisreg` de `Epi` toma la respuesta como `cbind(eventos, años-persona)` y modela la tasa directamente, sin *offset*.

```

m0 <- glm(cbind(event, lex.dur) ~ koen, family = poisreg, data = Rx)

ci.exp(m0) * c(1000, 1)

```

La primera fila es la tasa del nivel de referencia de `koen`, multiplicada por 1000 para dejarla por 1000 años-persona; debe coincidir con la tabla anterior. La segunda es la razón de tasas del otro nivel frente al de referencia, que no lleva escala.

Esta razón de tasas es cruda. Para ajustarla por edad basta con añadir la edad al modelo, y con eso ya se entra en Chapter 15..

15. Hacia dónde se generaliza esto

Passar por `Lexis` en lugar de tabular bandas de edad a mano compensa en cuanto el modelo crece:

- **Añadir la mortalidad** consiste en un segundo `cutLexis()` (o un tercer estado en `exit.status`). La muerte pasa entonces a competir con el diagnóstico, y `summary()/boxes()` informan de ambas transiciones sin ningún cambio en el código anterior.
- **Los efectos de periodo y cohorte** salen de dividir también en `P` y añadirlo al modelo; `apc.fit()` acepta directamente un objeto `Lexis` dividido.
- **Los desenlaces posdiagnóstico** se modelan en la escala `tfd` que `cutLexis()` ya creó: complicaciones, mortalidad, intensificación del tratamiento.
- **La prevalencia y la esperanza de vida** se obtienen del mismo objeto con `simLexis()`.
- **La entrada retardada** —la versión realista de esta cohorte— es un solo cambio en la lista `entry`; la división, el corte, la tabulación y el modelado no se tocan.

16. Errores frecuentes

- Dar a una escala de tiempo el nombre de una columna que ya existe. De ahí sale `Cannot merge data with duplicate names:age`. Renombrar la escala (A, P) o la columna.
- Dejar el tiempo-persona posdiagnóstico en el denominador. Hay que filtrar por `lex.Cst == "Healthy"` **después** de dividir, no antes de cortar.
- Pasar `sim_diab$age_dm` en lugar de `Lx$age_dm` a `cutLexis()`. Si al construir el objeto `Lexis` se descartó alguna fila, los cortes caen en silencio sobre las personas equivocadas.
- Cortar en un punto que cae fuera del seguimiento. `cutLexis()` no protesta: si el corte está en la entrada o antes, la persona empieza en el nuevo estado y su transición desaparece. Comparar el número de casos con el de transiciones es la forma más rápida de detectarlo.
- Derivar la edad al diagnóstico de `duration` en lugar de `stable_inclusion_date`. Solo coinciden si ambas variables comparten el mismo momento de referencia.
- Usar solo `lex.Xst == "T2D"` como indicador de evento. La forma segura es siempre `lex.Cst == "Healthy" & lex.Xst == "T2D"`.
- Mezclar unidades. Si la edad va en años y el calendario en días, `lex.dur` no significa nada. `cal.yr()` existe precisamente para evitarlo, y todo lo demás se mantiene en años de principio a fin.
- Olvidar que seguir desde el nacimiento es una comodidad de la simulación. En datos de registro reales fabrica tiempo-persona inmortal en las edades jóvenes.
- Graficar las tasas en un eje y lineal y concluir que los sexos divergen con la edad, cuando la razón de tasas es en realidad constante.

17.